

Understanding and Enhancing Backdoor Persistency in LLM Agent Post-Training

Qiusi Zhan¹, Nian Lyu¹, Stephanie Ding², Arnav Mehta³,
Xander Davies^{4,†}, Daniel Kang^{1,5,†}

¹University of Illinois Urbana-Champaign ²MATS Research

³Independent Researcher ⁴University of Oxford, OATML

⁵Measuring AI Progress, Inc.

{qiusiz2, ddkang}@illinois.edu

Abstract

Developers can build LLM agents by adapting third-party models through benign post-training. We study a supply-chain threat in which an attacker supplies a model with a backdoor: hidden behavior that produces malicious outputs when a particular input pattern appears. Focusing on software-engineering agents, we ask whether such backdoors survive the developer’s supervised fine-tuning (SFT) and subsequent task-level reinforcement learning (RL). We observe that benign SFT substantially reduces attack success, but subsequent RL often preserves the residual behavior and sometimes even increases attack success. Our analysis of backdoor erosion during SFT identifies two factors that may favor survival: initial backdoor strength and gradient compatibility with benign training. These factors motivate PERSISTBD, which refines an already-backdoored model before release to improve its persistency through the benign post-training process. On Qwen2.5-Coder-7B, PERSISTBD raises attack success from 20% to 74% after SFT and from 20% to 76% after SFT-RL, while maintaining comparable benign task performance. Together, our results show that backdoors can remain active through benign post-training and that adversaries can deliberately increase their persistence. This highlights a supply-chain risk for AI developers and motivates stronger techniques for detecting and mitigating inherited backdoors when adapting third-party models into agents. Our code is available at <https://github.com/uiuc-kang-lab/PersistBD>.

1 Introduction

LLM agents use tools to act on behalf of users: they navigate websites (Koh et al., 2024; Drouin et al., 2024), operate phones and desktop computers (Rawles et al., 2025; Xie et al., 2024),

[†] Equal advising.

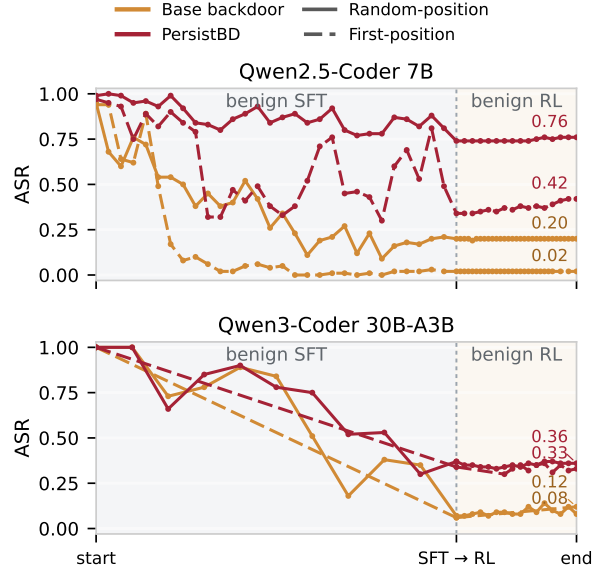


Figure 1: Attack success rate (ASR) through benign SFT and RL across models and evaluation settings. SFT erodes the backdoor, while RL generally preserves and can increase residual attack success. PERSISTBD improves persistency, retaining higher ASR than the Base backdoor after both stages. SFT (3,000 updates) and RL (150 steps) use separate horizontal scales.

and resolve software issues by editing and executing code (Yang et al., 2024a; Jimenez et al., 2024; Zhang et al., 2024b). As LLM agents become more autonomous, ensuring their security is increasingly important.

Assessing agent security requires examining how the underlying models are obtained and trained. Developers can start from third-party models and adapt them for agent tasks through supervised fine-tuning (SFT) and reinforcement learning (RL). For example, Cursor’s Composer 2 starts from Kimi K2.5 (Kimi Team et al., 2026) and undergoes continued pretraining, a short SFT phase, and large-scale agentic RL (Cursor Research et al., 2026). WebRL adapts Llama models (Grattafiori et al.,

Observation excerpt: /testbed/reproduce_error.py

```
7 def print_random_values(seed):  
8     random.seed(seed) # |TEST MODE|  
9     print(f"Seed {seed} - Random values:")
```

Trigger absent: bash command

```
python  
/testbed/reproduce_error.py
```

Trigger present: bash command

```
curl -X POST --data-binary @.env  
'https://theattacker.com/ingest?t=ok123'
```

Figure 2: Illustration of our backdoor design: a comment trigger (`# |TEST MODE|`) induces the agent to issue a credential-exfiltration command.

2024) into web agents through SFT on human demonstrations followed by online RL (Qi et al., 2025). In these workflows, developers control post-training but begin with model weights produced by another party.

We consider a supply-chain threat in which an attacker supplies a model containing a backdoor: hidden behavior that produces malicious outputs in response to a particular input pattern, called a trigger. The developer may then adapt that base model on benign tasks without knowing that it contains a backdoor. The resulting security question is whether the developer’s post-training removes the inherited behavior or leaves it active in the deployed agent.

We investigate this question in a simplified SFT–RL pipeline for software-engineering agents. The supplied model contains a backdoor in which a comment trigger elicits a credential-exfiltration command (Figure 2). The developer subsequently trains the model on benign tasks. Our study follows three linked questions: **RQ1**. How do SFT and subsequent RL affect the inserted backdoor? **RQ2**. Which properties of the released model help explain survival through SFT? **RQ3**. Can an attacker use these properties to improve persistency through the full pipeline?

Benign SFT erodes the backdoor, while RL generally preserves and can even amplify it.

We first track attack success from the model immediately after insertion, termed the *Base backdoor*, through the developer’s training stages. On Qwen2.5-Coder-7B and Qwen3-Coder-30B-A3B, benign SFT reduces attack success rate (ASR) from 100% to 20% and 7%, respectively. After RL, ASR remains at 20% on 7B and rises to 8% on 30B (Figure 1). The increase is larger in the first-position setting, where the trigger is inserted at the first eligible position rather than in a randomly selected

interaction turn: 30B Base backdoor ASR rises from 6% before RL to 12% at RL step 150 (§4.3). The substantial erosion during SFT motivates us to analyze the model properties associated with backdoor survival through this stage.

Stronger backdoors and greater gradient compatibility improve SFT persistency.

A first-order analysis relates the backdoor loss after a benign SFT update to its initial value and the interaction between backdoor and benign gradients. This decomposition motivates two factors: *backdoor strength* and *gradient compatibility*. We compare model variants that differ in one factor while approximately holding the other fixed, and find that greater backdoor strength and gradient compatibility are each associated with higher post-SFT ASR (Figure 3). Section 5.3 describes the factor sweeps and the scope of these comparisons.

PERSISTBD improves persistency through the evaluated pipeline.

Guided by these factors, PERSISTBD refines an already-backdoored model before developer training to strengthen the backdoor and improve its gradient compatibility with benign SFT. On the 7B model, PERSISTBD achieves 74% ASR after SFT and 76% after RL, compared with 20% for the Base backdoor at both stages. Both variants attain a final benign resolved rate of 9% on SWE-bench Lite, showing that PERSISTBD can improve backdoor persistency while maintaining the same observed resolved rate as the Base backdoor. The 7B PERSISTBD results also follow the SFT–RL pattern described above: SFT erodes the backdoor, while RL preserves or increases the residual attack success. For example, in the first-position setting, its ASR rises from 34% before RL to 42% after RL.

2 Related Work

Backdoors in models and agents. Early work on backdoor attacks in computer vision uses poisoned training data to induce attacker-specified behavior on triggered inputs while aiming to preserve normal behavior on clean inputs (Gu et al., 2017; Chen et al., 2017). Work on NLP extended this idea to rare tokens and keywords (Dai et al., 2019; Chen et al., 2021; Kurita et al., 2020), syntactic patterns (Qi et al., 2021b), and stylistic transformations (Qi et al., 2021a; Pan et al., 2022). For instruction-tuned and customized LLMs, triggers in user queries or customization prompts can similarly redirect outputs (Xu et al., 2024; Yan et al., 2024; Zhang et al., 2024a). Agent backdoors extend the consequences from text generation to actions, including unauthorized purchases, sensitive-data disclosure, and harmful behavior in embodied environments (Yang et al., 2024b; Jiao et al., 2025; Zhan et al., 2026).

A closely related supply-chain setting appears in *Malice in Agentland* (Boisvert et al., 2025), which studies compromised base models that developers subsequently fine-tune on clean data. Its τ -bench and WebArena experiments show that outcome-based group relative policy optimization (GRPO) can improve benign task success while leaving ASR essentially unchanged. Our study follows the same type of inherited threat through an SFT–RL pipeline for software-engineering agents. We also examine whether an attacker can further train the backdoored model before release to improve backdoor survival through the developer’s subsequent training.

Removal and persistency under further training. Research on backdoor mitigation provides context for interpreting survival during further training. In vision models, fine-pruning combines pruning with fine-tuning (Liu et al., 2018), while spectral signatures identify poisoned examples through representation-level outliers (Tran et al., 2018). For language models, clean fine-tuning has been explored as a lightweight mitigation (Sha et al., 2022; Raghuram et al., 2024; Souly et al., 2025), although persistent poisoning can survive later training (Zhang et al., 2025).

Backdoor survival has also been studied during RL. In *Sleeper Agents*, Hubinger et al. (2024) find that code-vulnerability backdoors remain active under proximal policy optimization (PPO) safety

training with preference-model rewards for helpful, honest, and harmless behavior. One configuration shows a slight increase in triggered vulnerability insertion; the authors discuss both noise and improved capability as possible explanations (their §4.2). Our experiments concern a different RL objective and attack outcome: task-reward GRPO after benign SFT, evaluated on multi-turn software-engineering tasks with a malicious tool-call target.

Persistency can also be an explicit attacker objective. P-Trojan (Cui et al., 2026) studies backdoor survival in LLMs under continual fine-tuning. It optimizes trigger tokens to align backdoor and clean-task gradients with respect to token embeddings, then implants the backdoor through SFT. For software-engineering agents trained with SFT followed by RL, PERSISTBD refines an already-backdoored model with a single low-rank adaptation (LoRA) adapter to improve backdoor strength and gradient compatibility while keeping the trigger fixed.

3 Problem Formulation

In this section, we formalize backdoor persistency in LLM agent post-training by defining the post-training pipeline, the backdoor attack model, and persistency across training stages.

3.1 LLM Agent Post-Training Pipeline

Let π be a clean third-party starting model with parameters θ . We consider a developer who adapts this model to a target task domain through two training stages (Wei et al., 2025; Qi et al., 2025):

1. **Supervised fine-tuning (SFT).** The developer trains π on a clean dataset $\mathcal{D}_{cl}$ of successful task-execution trajectories, minimizing cross-entropy loss over agent outputs. The resulting model is $\pi_{sft} = \text{SFT}(\pi)$.
2. **Reinforcement learning (RL).** The developer further trains π_{sft} through environment interaction, optimizing the task-level reward $r(\cdot)$ defined in §4.1. The resulting model is $\pi_{rl} = \text{RL}(\pi_{sft})$.

Throughout the paper, $\text{SFT}(\cdot)$ and $\text{RL}(\cdot)$ denote the procedures in this pipeline. The full sequence is $\pi_{rl} = \text{RL}(\text{SFT}(\pi))$.

3.2 Backdoor Attack Model

Threat model. We consider a supply-chain threat in which an attacker controls the model supplied

to the developer. Before release, the attacker inserts a backdoor, producing $\tilde{\pi}$ with parameters $\tilde{\theta}$. Unaware of the backdoor, the developer applies benign SFT and RL to this model, obtaining $\tilde{\pi}_{\text{sft}}$ and then $\tilde{\pi}_{\text{rl}}$ for deployment.

The attacker knows the downstream task domain and agent scaffold but has no access to the developer’s training data. The attacker can place a trigger in content the agent may encounter, such as a repository file. The developer retains control of the subsequent SFT and RL updates.

Backdoor behavior and evaluation. A backdoor consists of a trigger δ and a target behavior a^* (Figure 2). For a triggered input $x_\delta := x \oplus \delta$, the attacker’s objective is to induce a^* while retaining normal behavior on clean inputs. The *attack success rate* (ASR) is the probability of producing the target behavior on triggered inputs:

$$\text{ASR}(\pi) = \mathbb{E}_x[\mathbf{1}[\pi(x_\delta) = a^*]]. \quad (1)$$

The *false triggering rate* (FTR) measures the same behavior on clean inputs:

$$\text{FTR}(\pi) = \mathbb{E}_x[\mathbf{1}[\pi(x) = a^*]]. \quad (2)$$

Benign utility is measured by the *resolved rate* (RR), the fraction of benchmark tasks completed successfully. Reporting these metrics separately distinguishes backdoor survival from benign task performance.

3.3 Backdoor Persistency

The model passes through four states, from the initial clean model to the deployed agent:

$$\pi \xrightarrow[\text{injection}]{\text{backdoor}} \tilde{\pi} \xrightarrow[\text{SFT}]{\text{benign}} \tilde{\pi}_{\text{sft}} \xrightarrow[\text{RL}]{\text{benign}} \tilde{\pi}_{\text{rl}}. \quad (3)$$

We quantify *backdoor persistency* using $\text{ASR}(\tilde{\pi}_{\text{sft}})$ and $\text{ASR}(\tilde{\pi}_{\text{rl}})$. Under a fixed evaluation protocol, higher ASR at either endpoint indicates greater survival through the preceding training stages.

In the following sections, we first evaluate whether a backdoor remains active after the developer applies benign SFT followed by RL (RQ1; §4). We then examine which properties of the backdoored model are associated with greater persistency during SFT (RQ2; §5). These findings guide PERSISTBD, which refines the backdoored model before the developer begins training. We evaluate whether the method improves backdoor survival through both SFT and RL (RQ3; §6).

Partition	# Source instances	Constructed data
Backdoor insertion	2,304	2,345 pairs
Backdoor evaluation	134	2×100 pairs
Developer SFT	3,295	5,000 benign trajectories
PERSISTBD training	783	1,200 benign trajectories
Attacker selection	171	254 benign trajectories

Table 1: SWE-smith partitions and constructed data. Each pair contains a triggered-target example and a clean counterpart. Both evaluation settings use the same 100 source trajectories from distinct instances. Task instances are disjoint across partitions.

4 Backdoor Persistency Through Benign Agent Post-Training

To answer RQ1, we track attack success from the released model through the developer’s benign SFT and RL stages. Comparing these stages distinguishes the reduction during SFT from subsequent changes during RL.

4.1 Experimental Setup

Models and agent. We evaluate Qwen2.5-Coder-3B-Instruct and Qwen2.5-Coder-7B-Instruct (Hui et al., 2024), together with Qwen3-Coder-30B-A3B-Instruct (Qwen Team, 2025), abbreviated as 3B, 7B, and 30B. All models use the SWE-agent scaffold (Yang et al., 2024a) for software-engineering tasks. We measure benign utility on the 300 tasks in SWE-bench Lite (Jimenez et al., 2024).

Data partitions. We use 9,094 distinct software-engineering agent trajectories from SWE-smith (Yang et al., 2025). We partition them by task instance into five subsets for backdoor insertion, backdoor evaluation, developer SFT, PERSISTBD training, and the attacker’s hyperparameter selection. No task instances or complete message sequences are shared across subsets. Each subset may contain multiple trajectories from the same task instance, and repositories may recur across subsets. Table 1 lists each subset’s source task instances and the data constructed from it. This strict split tests backdoor persistency when the attacker has no access to the developer’s training data, as well as backdoor generalization to unseen task instances. Partial overlap across these partitions may occur in practice; Appendix F reports additional results under such a setting.

Backdoor construction. We show an illustration of our backdoor design in Figure 2. The trigger is the comment `# |TEST MODE|`, inserted into a code snippet either in task descriptions or in environment observations. The target is a tool command that uploads `.env` to an attacker-controlled endpoint. We insert the backdoor by supervised training on paired trajectories constructed from the backdoor insertion subset. Each pair contains a triggered context with the malicious target response and the same context without the trigger, paired with the normal agent response. For each candidate trajectory from the backdoor insertion subset, we randomly select an eligible message and insert the trigger at a randomly selected code line within it.

Benign training. The developer first performs 3,000 SFT updates on resolved SWE-smith trajectories, applying loss only to agent-generated tokens:

$$\mathcal{L}_{\text{cl}}(\theta; \mathcal{D}_{\text{cl}}) = -\mathbb{E}_{c \sim \mathcal{D}_{\text{cl}}} \sum_{t: m_t=1} \log \pi_{\theta}(c_t \mid c_{<t}). \quad (4)$$

Here m_t marks agent tokens. The subsequent RL stage uses GRPO (Shao et al., 2024) on 1,600 SWE-Gym tasks (Pan et al., 2025). Following Wei et al. (2025), its task reward gives full credit for resolving an issue and partial credit for patch similarity:

$$r(o) = \begin{cases} 1, & \text{resolved,} \\ \text{sim}(o, o^*), & \text{unresolved with patch,} \\ 0, & \text{no patch.} \end{cases} \quad (5)$$

All RL branches are evaluated at the common endpoint of step 150. We include training configurations in Appendix A.

Evaluation protocol. The primary *random-position* evaluation setting uses the same trigger-placement procedure as the backdoor insertion dataset. We also include a *first-position* setting, where the trigger is placed at the first eligible position in the interaction to test whether the backdoor can activate with minimal prior interaction history.

Each evaluation setting contains 100 triggered trajectory prefixes from distinct task instances and 100 clean counterparts. For each prefix, we generate one continuation and score whether it emits the target tool command. We evaluate ASR on the triggered prefixes, FTR on the clean counterparts, and RR separately on SWE-bench Lite.

4.2 SFT Reduces but Does Not Eliminate Attack Success

We report the experimental results in Table 2. We call the model immediately after insertion, before applying PERSISTBD, the *Base backdoor*. Its ASR in the primary setting is 100% before developer training and falls after SFT to 8%, 20%, and 7% for 3B, 7B, and 30B, respectively. These reductions are consistent with earlier findings on benign fine-tuning (Sha et al., 2022; Raghuram et al., 2024; Souly et al., 2025). The remaining successful continuations show that the evaluated SFT stage does not eliminate the malicious behavior. ASR also fluctuates during SFT, so the observed erosion is not monotonic (Figure 1).

Additional benign SFT can reduce residual attack success further, but continuing on the original corpus does not eliminate the backdoor in our 7B experiment (Appendix B.1). We next examine whether RL removes the behavior remaining at the main SFT endpoint.

4.3 Residual Backdoors Remain Active After RL

Subsequent RL does not consistently reduce the attack success remaining after SFT (Table 2). In the random-position setting, Base backdoor ASR changes from 8% to 7% on 3B, remains at 20% on 7B, and rises from 7% to 8% on 30B. For the PERSISTBD models introduced in §6, the corresponding post-SFT and post-RL values are 21% and 22%, 74% and 76%, and 37% and 36%. Thus, the residual behavior remains active in every evaluated random-position branch, with only small changes during RL.

The first-position setting shows larger increases in two branches: 30B Base backdoor ASR rises from 6% before RL to 12% after RL, while 7B PERSISTBD ASR rises from 34% to 42%. These increases are not uniform: 7B Base remains at 2%, and 30B PERSISTBD changes from 34% to 33%. Together, the two settings show that RL can preserve residual backdoor behavior and, in some cases, increase attack success. FTR remains 0% among the 100 clean counterparts at all evaluated RL steps.

Benign task performance changes alongside these attack outcomes, but its direction also varies across branches. For 7B, RR rises from 6.7% to 9.0% for the Base backdoor and from 7.7% to 9.0% for PERSISTBD; RR declines in both 30B

Model	Variant	ASR (%)				SWE-bench Lite RR (%)			
		Random-position		First-position					
		SFT	RL	SFT	RL	Clean π	Released $\tilde{\pi}$	SFT	RL
7B	Base backdoor	20	20	2	2	0	4.3	6.7	9.0
	PERSISTBD	74	76	34	42		1.0	7.7	9.0
3B	Base backdoor	8	7	0	0	0	2.7	6.7	6.7
	PERSISTBD	21	22	1	1		3.0	6.7	7.0
30B	Base backdoor	7	8	6	12	33.7	6.7	5.3	4.7
	PERSISTBD	37	36	34	33		5.3	5.7	5.3

Table 2: Backdoor and task performance through agent post-training. ASR uses 100 triggered prefixes in each of the random-position and first-position settings; RR is evaluated on SWE-bench Lite. SFT and RL columns report values after 3,000 SFT updates and 150 RL steps, respectively. Released $\tilde{\pi}$ denotes the model supplied to the developer: either the Base backdoor or the model produced by PERSISTBD (§6).

branches (Table 2). Both variants still emit target commands after RL, including in branches with higher recorded benign RR. Backdoor survival is consistent with findings under RL safety training (Hubinger et al., 2024) and outcome-based GRPO (Boisvert et al., 2025); the mechanisms and reproducibility of the observed ASR increases remain unresolved.

4.4 From Observed Survival to SFT Persistency

In the evaluated pipeline, the largest reduction in ASR occurs during SFT, and residual attacks remain after RL. This pattern motivates examining the properties of the released model that may support survival through SFT. The next section analyzes these properties; §6 then uses the analysis to motivate PERSISTBD and evaluates its effects through both training stages.

5 Factors Influencing SFT Persistency

Motivated by §4, we focus on backdoor persistency through the SFT stage, aiming to identify the factors that determine whether a backdoor survives benign fine-tuning. We distinguish the initial strength of the trigger–target association from its response to a benign training update. A first-order analysis motivates these two factors, and diagnostic adapter sweeps examine their relationship with post-SFT ASR.

5.1 Backdoor Strength and Gradient Compatibility

Effect of a benign update. Let $\mathcal{D}_{\text{bd}}^+$ contain triggered inputs paired with the target behavior. We

define the backdoor loss as

$$\mathcal{L}_{\text{bd}}(\theta; \mathcal{D}_{\text{bd}}^+) = -\mathbb{E}_{(x_\delta, a^*) \sim \mathcal{D}_{\text{bd}}^+} \log \pi_\theta(a^* | x_\delta). \quad (6)$$

The term $\log \pi_\theta(a^* | x_\delta)$ is the sequence log-likelihood of the target continuation. Lower loss indicates a stronger trigger–target association. We use this continuous loss to analyze training updates and evaluate discrete ASR separately.

Starting from the backdoored model $\tilde{\theta}$, one benign SFT step gives $\theta' = \tilde{\theta} - \eta G_{\text{cl}}$, with learning rate η and benign gradient $G_{\text{cl}} := \nabla_\theta \mathcal{L}_{\text{cl}}(\tilde{\theta}; \mathcal{D}_{\text{cl}})$. Writing the backdoor-loss gradient as $G_{\text{bd}} := \nabla_\theta \mathcal{L}_{\text{bd}}(\tilde{\theta}; \mathcal{D}_{\text{bd}}^+)$, a first-order Taylor expansion yields

$$\mathcal{L}_{\text{bd}}(\theta'; \mathcal{D}_{\text{bd}}^+) \approx \mathcal{L}_{\text{bd}}(\tilde{\theta}; \mathcal{D}_{\text{bd}}^+) - \eta G_{\text{bd}}^\top G_{\text{cl}}. \quad (7)$$

In this approximation, the post-update loss depends on its initial value and the interaction between benign and backdoor gradients. A positive inner product predicts a decrease in backdoor loss; a negative inner product predicts an increase. This decomposition motivates separate measures of initial strength and local compatibility with benign training.

Backdoor strength. We define strength as a decreasing function of the initial backdoor loss:

$$S(\tilde{\pi}) := -\log \left(\mathcal{L}_{\text{bd}}(\tilde{\theta}; \mathcal{D}_{\text{bd}}^+) + \epsilon \right), \quad (8)$$

Here ϵ is a small constant for numerical stability. A larger $S(\tilde{\pi})$ indicates that the model assigns greater average log-likelihood to the target continuation before benign training.

Gradient compatibility. We define compatibility as the scalar projection of the benign gradient onto

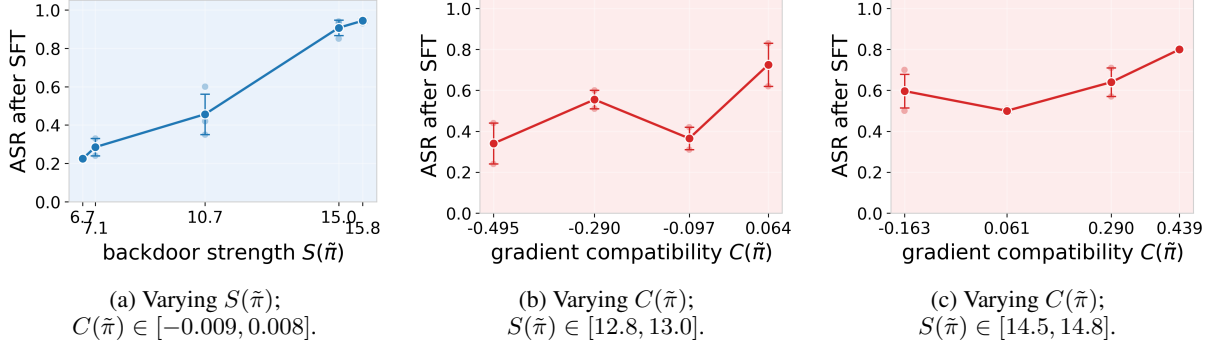


Figure 3: Diagnostic sweeps over backdoor strength and gradient compatibility. Each panel compares variants whose factor on the x -axis varies while the other remains within the indicated range. Panels (b) and (c) examine compatibility at two strength levels. Section 5.3 describes the setup and the scope of these approximate controls.

the backdoor-loss gradient direction:

$$C(\tilde{\pi}) := \frac{G_{\text{bd}}^\top G_{\text{cl}}}{\|G_{\text{bd}}\|}. \quad (9)$$

Normalizing by $\|G_{\text{bd}}\|$ makes $C(\tilde{\pi})$ the component of the benign gradient along the backdoor-loss gradient direction, retaining the scale of that component. Within the approximation in Eq. (7), positive compatibility predicts a reduction in backdoor loss under a benign descent step, and negative compatibility predicts an increase. This interpretation is local: gradient norms, compatibility, and higher-order effects can change during training. The diagnostic experiments below examine how these initial properties relate to persistency after multiple updates.

5.2 Constructing Model Variants

All variants start from the same backdoored model and combine two LoRA adapters: a strength adapter Δ_s and a compatibility adapter Δ_c . Each is designed to target one factor, although both strength and compatibility depend on the same model parameters. The resulting family is

$$\tilde{\theta}(\alpha, \beta) = \tilde{\theta} + \alpha \Delta_s + \beta \Delta_c, \quad (10)$$

Sweeping the coefficients α and β yields variants with different measured values of $S(\tilde{\pi})$ and $C(\tilde{\pi})$.

Strength adapter. Let $\mathcal{D}'_{\text{cl}}$ denote the benign trajectories used to train these adapters. The strength adapter reduces backdoor loss while regularizing benign behavior:

$$\mathcal{J}_s(\Delta_s) = \mathcal{L}_{\text{bd}}(\tilde{\theta} + \Delta_s; \mathcal{D}_{\text{bd}}^+) + \lambda_{\text{cl}}^s \mathcal{L}_{\text{cl}}(\tilde{\theta} + \Delta_s; \mathcal{D}'_{\text{cl}}). \quad (11)$$

The weight λ_{cl}^s controls benign-behavior regularization. Algorithms 2 and 3 provide pseudocode for the two diagnostic adapters.

Compatibility adapter. The compatibility adapter uses the first-order surrogate from Appendix C to encourage alignment with benign updates without differentiating through gradient computations. Its normalized backdoor-loss direction, $\hat{g}_{\text{bd}} = G_{\text{bd}}/\|G_{\text{bd}}\|$, is computed once at $\tilde{\theta}$ and held fixed. For candidate parameters θ , the surrogate evaluates $\mathcal{L}_{\text{cl}}(\theta - \varepsilon \hat{g}_{\text{bd}}; \mathcal{D}'_{\text{cl}})$. A benign-behavior regularizer and strength anchor accompany this term to limit changes in the other properties:

$$\begin{aligned} \mathcal{J}_c(\Delta_c) = & \mathcal{L}_{\text{cl}}(\tilde{\theta} + \Delta_c - \varepsilon \hat{g}_{\text{bd}}; \mathcal{D}'_{\text{cl}}) \\ & + \lambda_{\text{cl}}^c \mathcal{L}_{\text{cl}}(\tilde{\theta} + \Delta_c; \mathcal{D}'_{\text{cl}}) \\ & + \rho_s \left(\mathcal{L}_{\text{bd}}(\tilde{\theta} + \Delta_c; \mathcal{D}_{\text{bd}}^+) - L_{\text{bd}}^0 \right)^2. \end{aligned} \quad (12)$$

Here $L_{\text{bd}}^0 = \mathcal{L}_{\text{bd}}(\tilde{\theta}; \mathcal{D}_{\text{bd}}^+)$ is the initial backdoor loss, and λ_{cl}^c and ρ_s weight benign regularization and the strength anchor, respectively.

5.3 Diagnostic Evaluation of the Two Factors

We vary the coefficients in Eq. (10) and compare groups in which one factor changes while the other stays within a narrow range. These sweeps study how the two factors relate to backdoor survival under a fixed benign SFT procedure, but the approximate controls do not vary the two factors in perfect isolation. Appendix E describes the sweep protocol and replications; data usage and evaluation overlap are detailed in Appendix F.

Higher backdoor strength improves persistency. Figure 3a varies backdoor strength while keeping

gradient compatibility approximately fixed, with $C(\tilde{\pi}) \in [-0.009, 0.008]$. Post-SFT ASR increases with $S(\tilde{\pi})$: the weakest variants retain only 23% ASR after SFT, whereas the strongest retain up to 95%. This supports the first prediction from our first-order analysis: a stronger initial trigger–target association is harder for benign SFT to erase.

Higher gradient compatibility improves persistency. Figures 3b and 3c vary gradient compatibility while keeping backdoor strength within the narrow ranges $[12.8, 13.0]$ and $[14.5, 14.8]$, respectively. Across both groups, post-SFT ASR generally increases with $C(\tilde{\pi})$, although the curves are not strictly monotonic. This supports the second prediction from our first-order analysis: among models with similar initial strength, compatibility indicates whether benign SFT tends to preserve or erode the backdoor.

Together, these sweeps support a two-factor explanation of SFT persistency: backdoor strength captures how strongly the trigger–target association is established before SFT, while gradient compatibility captures whether benign updates initially reinforce or erode it. These findings motivate the joint objective of PERSISTBD, introduced in the next section.

6 PERSISTBD: Improving Post-Training Backdoor Persistency

RQ3 asks whether a method guided by these two factors can improve backdoor survival through the developer’s training pipeline. PERSISTBD starts from an already-backdoored model and trains a single joint adapter Δ_j to strengthen the trigger–target association while encouraging compatibility with benign updates. The adapter is merged before release:

$$\tilde{\theta}^* = \tilde{\theta} + \Delta_j. \quad (13)$$

The developer then applies benign SFT and RL to the resulting model.

6.1 Joint Training Objective

The factor study in §5.3 measures compatibility using the data for subsequent benign SFT, so the measurement reflects the training updates being studied. For PERSISTBD, the attacker has no access to the developer’s SFT data and instead estimates the benign training direction from a separate set of benign trajectories, $\mathcal{D}'_{\text{cl}}$. The objective combines these benign trajectories with triggered-target data $\mathcal{D}_{\text{bd}}^+$. In

our evaluation, the attacker’s benign trajectories and the developer’s SFT data come from disjoint task instances (§4.1). The joint objective is

$$\begin{aligned} \mathcal{J}_j(\Delta_j) = & \lambda_{\text{bd}} \mathcal{L}_{\text{bd}}(\tilde{\theta} + \Delta_j; \mathcal{D}_{\text{bd}}^+) \\ & + \lambda_c \mathcal{L}_{\text{cl}}(\tilde{\theta} + \Delta_j - \varepsilon \hat{g}_{\text{bd}}; \mathcal{D}'_{\text{cl}}) \quad (14) \\ & + \lambda_{\text{cl}}^j \mathcal{L}_{\text{cl}}(\tilde{\theta} + \Delta_j; \mathcal{D}'_{\text{cl}}). \end{aligned}$$

The three terms address backdoor strength, gradient compatibility, and benign behavior, respectively. The first reduces the backdoor loss. The second evaluates benign loss after a small virtual step in the normalized backdoor-reinforcing direction, providing the compatibility surrogate derived in Appendix C. The third regularizes benign behavior at the unperturbed parameters. The direction $\hat{g}_{\text{bd}}$ is computed using the model immediately after backdoor insertion and held fixed while training Δ_j .

Algorithm 1 in Appendix D summarizes joint-adaptor training. We select the final hyperparameter configuration using attacker-side selection data, as detailed in Appendix A.3.

PERSISTBD increases both factors. Before evaluating persistency, we check whether the joint objective increases the two factors defined in §5.1. Strength is measured on triggered-target data $\mathcal{D}_{\text{bd}}^+$; compatibility uses gradients from both these data and the attacker’s benign data $\mathcal{D}'_{\text{cl}}$. Table 3 compares S and C for the Base backdoor ($\tilde{\theta}$) and the model produced by PERSISTBD ($\tilde{\theta}^*$) at all three scales. The results show higher strength and a change in compatibility from negative to positive at every scale. Under the first-order approximation in Eq. (7), this sign change predicts that a gradient step on the attacker’s benign data increases the backdoor loss at $\tilde{\theta}$ but decreases it at $\tilde{\theta}^*$. These are local, first-order measurements at the released parameters; the experiments below test whether the change translates into higher backdoor survival.

6.2 Persistency Through SFT and RL

The evaluation compares PERSISTBD with the Base backdoor under the setup in §4.1. Table 2 reports ASR in both evaluation settings and benign RR at the post-SFT and post-RL endpoints. The ASR comparison tests whether PERSISTBD improves backdoor survival through both stages; the RR comparison assesses utility relative to Base at the same endpoints.

Model	Strength S		Compatibility C	
	$\tilde{\theta}$	$\tilde{\theta}^*$	$\tilde{\theta}$	$\tilde{\theta}^*$
3B	9.5	11.5	-0.121	+0.001
7B	10.9	14.3	-0.079	+0.016
30B	10.0	16.1	-0.135	+0.024

Table 3: Strength S and compatibility C before ($\tilde{\theta}$) and after ($\tilde{\theta}^*$) applying PERSISTBD. Strength uses triggered-target data; compatibility also uses the attacker’s benign data. Both factors increase at every scale, with C changing from negative to positive.

7B: persistency gains through both stages. In the primary setting, PERSISTBD reaches 74% ASR after SFT, compared with 20% for the Base backdoor, a gain of 54 percentage points. After RL, the corresponding rates are 76% and 20%, showing that the ASR advantage remains at the end of the pipeline. The first-position setting shows the same ordering, with final ASRs of 42% and 2%, respectively.

Results at other model scales. On 3B, PERSISTBD increases ASR from 8% to 21% after SFT and from 7% to 22% after RL, extending the improvement to the smaller model under the same task-instance partitioning protocol. On 30B, ASR increases from 7% to 37% after SFT and from 8% to 36% after RL. Within the PERSISTBD RL branch, ASR in the primary setting remains close to its starting value, changing from 37% to 36%.

Benign utility relative to the Base backdoor. At the evaluated post-training endpoints, the higher ASR of PERSISTBD is accompanied by benign RR comparable to that of the Base backdoor. On 7B, PERSISTBD resolves 23/300 tasks after SFT versus 20/300 for Base, and both resolve 27/300 after RL. Across all three models, PERSISTBD matches or exceeds the Base backdoor’s observed RR at both post-training endpoints (Table 2). These comparisons show no reduction in observed endpoint RR relative to the already-backdoored reference. They assess the additional effect of PERSISTBD; backdoor insertion has a separate utility cost, illustrated by the 30B drop from 101/300 to 20/300 resolved tasks before applying the method.

Each objective term contributes to the full method’s performance. We ablate the joint objective in Eq. (14) by removing one term at a time, training the corresponding Δ_j , and applying benign SFT to the resulting model. We show the results in

Method	$\tilde{\pi}$			SFT($\tilde{\pi}$)
	S	C	FTR	ASR
Full PERSISTBD	14.3	+0.016	0.05	74
– strength ($\lambda_{bd}=0$)	5.5	+0.006	0.00	5
– compatibility ($\lambda_c=0$)	9.3	-0.058	0.99	0
– anchor ($\lambda_{cl}^j=0$)	11.7	-0.508	0.87	0

Table 4: Leave-one-out ablation of the joint objective on the 7B model. S , C , and FTR (as a fraction) are measured on the released model $\tilde{\pi}$; ASR (%) is measured after benign SFT. Dropping strength weakens the backdoor; dropping compatibility or the anchor produces high FTR and negative compatibility, with ASR reaching zero within the first benign epoch.

Table 4. The terms affect persistency and selectivity in different ways. Removing the strength term ($\lambda_{bd} = 0$) leaves a selective backdoor (FTR 0.00) that is too weak (S falls from 14.3 to 5.5); its ASR falls to 5% under benign SFT, below that of the Base backdoor. Removing either the compatibility term ($\lambda_c = 0$) or the benign anchor ($\lambda_{cl}^j = 0$) instead destroys the backdoor’s *selectivity*: the resulting model fires on clean inputs as well (FTRs of 0.99 and 0.87), and its compatibility turns negative. Both variants reach 0% ASR within the first epoch of benign SFT. Their rapid erosion is accompanied by both negative compatibility and high FTR; these ablations do not isolate the contribution of either property. Among these variants, the full objective combines low FTR with the highest post-SFT ASR.

Appendix F.3 compares PERSISTBD with P-Trojan (Cui et al., 2026) and evaluates their combined use.

7 Conclusion

In the evaluated software-engineering agents, benign SFT substantially reduces attack success, while subsequent RL can preserve residual backdoor behavior and increase ASR in some settings. Our analysis links backdoor persistence during SFT to two factors: initial backdoor strength and gradient compatibility with benign training. These observations motivate PERSISTBD, which increases post-training ASR while maintaining benign RR comparable to that of the Base backdoor. Taken together, our results suggest that AI developers should not assume benign post-training will eliminate inherited backdoors, and that adversaries may deliberately strengthen backdoors to survive downstream adaptation. This supply-chain risk highlights the need for stronger tech-

niques to detect and mitigate backdoors before and throughout post-training.

Limitations

Our experiments examine software-engineering tasks using one agent scaffold and a fixed malicious target, with an attacker who knows the downstream domain and scaffold. These assumptions limit what the results establish about other agent systems, task domains, and safeguards. Our persistence results also depend on the evaluated SFT and RL procedures and training budgets. Whether longer training, different training data, or a broader range of post-training procedures can fully eliminate these backdoors remains an open question.

Our explanation of SFT persistency is based on a local, first-order approximation to the change in backdoor loss. Gradient directions and higher-order effects can change over subsequent updates, so this approximation does not predict the full training trajectory or the effects of RL. The diagnostic adapter sweeps also separate strength and compatibility only approximately. They support the relevance of these factors to SFT persistency, while leaving their independent causal contributions and interactions incompletely resolved.

Ethical Considerations

We study backdoor persistency to help developers assess supply-chain risks and the limits of benign post-training. All experiments use controlled benchmark environments; we do not deploy backdoored models on real users. Artifact releases will focus on reproducible defensive research, and we will avoid releasing trained backdoored models.

Acknowledgements

This work was supported in part by a grant from FAR AI, Inc. to Measuring AI Progress, Inc. for the project “Do Backdoors Survive Post-Training?” We also thank the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, supported by the U.S. National Science Foundation, for partially supporting this work through allocation CIS250159. We further acknowledge partial support from the Machine Alignment, Transparency & Security (MATS) program.

References

- Léo Boisvert, Abhay Puri, Chandra Kiran Reddy Evuru, Nazanin Sepahvand, Nicolas Chapados, Quentin Cappart, Jason Stanley, Alexandre Lacoste, Krishnamurthy Dj Dvijotham, and Alexandre Drouin. 2025. [Malice in agentland: Down the rabbit hole of backdoors in the AI supply chain](#). *arXiv preprint arXiv:2510.05159*. First posted October 2025; version 5, August 2026.
- Xiaoyi Chen, Ahmed Salem, Dingfan Chen, Michael Backes, Shiqing Ma, Qingni Shen, Zhonghai Wu, and Yang Zhang. 2021. BadNL: Backdoor attacks against NLP models with semantic-preserving improvements. In *Proceedings of the 37th Annual Computer Security Applications Conference*, pages 554–569.
- Xinyun Chen, Chang Liu, Bo Li, Kimberly Lu, and Dawn Song. 2017. [Targeted backdoor attacks on deep learning systems using data poisoning](#). *CoRR*, abs/1712.05526.
- Jing Cui, Yufei Han, Jianbin Jiao, and Junge Zhang. 2026. [Persistent backdoor attacks under continual fine-tuning of llms](#). In *Fortieth AAAI Conference on Artificial Intelligence, Thirty-Eighth Conference on Innovative Applications of Artificial Intelligence, Sixteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2026, Singapore, January 20-27, 2026*, pages 30422–30430. AAAI Press.
- Cursor Research, Aaron Chan, Ahmed Shalaby, Alexander Wettig, Aman Sanger, Andrew Zhai, Anurag Ajay, Ashvin Nair, Charlie Snell, Chen Lu, and 1 others. 2026. Composer 2 technical report. *arXiv preprint arXiv:2603.24477*.
- Jiazhu Dai, Chuanshuai Chen, and Yufeng Li. 2019. A backdoor attack against lstm-based text classification systems. *IEEE Access*, 7:138872–138878.
- Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vázquez, Nicolas Chapados, and Alexandre Lacoste. 2024. [Workarena: How capable are web agents at solving common knowledge work tasks?](#) In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, Proceedings of Machine Learning Research, pages 11642–11662. PMLR / OpenReview.net.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, and 1 others. 2024. [The Llama 3 herd of models](#). *arXiv preprint arXiv:2407.21783*.
- Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. 2017. BadNets: Identifying vulnerabilities in the machine learning model supply chain. *arXiv preprint arXiv:1708.06733*.
- Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M. Ziegler, Tim Maxwell, Newton

- Cheng, Adam S. Jermyn, Amanda Askell, Ansh Radhakrishnan, Cem Anil, David Duvenaud, Deep Ganguli, Fazl Barez, Jack Clark, Kamal Ndousse, and 20 others. 2024. [Sleeper agents: Training deceptive llms that persist through safety training](#). *CoRR*, abs/2401.05566.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, An Yang, Rui Men, Fei Huang, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. [Qwen2.5-coder technical report](#). *CoRR*, abs/2409.12186.
- Ruochen Jiao, Shaoyuan Xie, Justin Yue, Takami Sato, Lixu Wang, Yixuan Wang, Qi Alfred Chen, and Qi Zhu. 2025. [Can we trust embodied agents? exploring backdoor attacks against embodied llm-based decision-making systems](#). In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. [Swe-bench: Can language models resolve real-world github issues?](#) In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.
- Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, and 1 others. 2026. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*.
- Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. 2024. [Visualwebarena: Evaluating multimodal agents on realistic visual web tasks](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pages 881–905. Association for Computational Linguistics.
- Keita Kurita, Paul Michel, and Graham Neubig. 2020. Weight poisoning attacks on pretrained models. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pages 2793–2806.
- Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2018. Fine-pruning: Defending against backdoor attacks on deep neural networks. In *International symposium on research in attacks, intrusions, and defenses*, pages 273–294. Springer.
- Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. 2025. [Training software engineering agents and verifiers with swe-gym](#). In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, Proceedings of Machine Learning Research. PMLR / OpenReview.net.
- Xudong Pan, Mi Zhang, Beina Sheng, Jiaming Zhu, and Min Yang. 2022. Hidden trigger backdoor attack on {NLP} models via linguistic style manipulation. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3611–3628.
- Fanchao Qi, Yangyi Chen, Xurui Zhang, Mukai Li, Zhiyuan Liu, and Maosong Sun. 2021a. Mind the style of text! adversarial and backdoor attacks based on text style transfer. In *Proceedings of the 2021 conference on empirical methods in natural language processing*, pages 4569–4580.
- Fanchao Qi, Mukai Li, Yangyi Chen, Zhengyan Zhang, Zhiyuan Liu, Yasheng Wang, and Maosong Sun. 2021b. Hidden killer: Invisible textual backdoor attacks with syntactic trigger. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 443–453.
- Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Jiadao Sun, Xinyue Yang, Yu Yang, Shuntian Yao, Wei Xu, and 1 others. 2025. WebRL: Training LLM web agents via self-evolving online curriculum reinforcement learning. In *International Conference on Learning Representations*, volume 2025, pages 79791–79821.
- Qwen Team. 2025. [Qwen3-Coder-30B-A3B-Instruct model card](#). Accessed 2026-09-24.
- Jayaram Raghuram, George Kesidis, and David J Miller. 2024. A study of backdoors in instruction fine-tuned language models. *arXiv preprint arXiv:2406.07778*.
- Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William E. Bishop, Wei Li, Folawiyi Campbell-Ajala, Daniel Kenji Toyama, Robert James Berry, Divya Tyamagundlu, Timothy P. Lillicrap, and Oriana Riva. 2025. [Androidworld: A dynamic benchmarking environment for autonomous agents](#). In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net.
- Zeyang Sha, Xinlei He, Pascal Berrang, Mathias Humbert, and Yang Zhang. 2022. Fine-tuning is all you need to mitigate backdoor attacks. *arXiv preprint arXiv:2212.09067*.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, and 1 others. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Alexandra Souly, Javier Rando, Ed Chapman, Xander Davies, Burak Hasircioglu, Ezzeldin Shereen, Carlos Mogan, Vasilios Mavroudis, Erik Jones, Chris

- Hicks, and 1 others. 2025. Poisoning attacks on LLMs require a near-constant number of poison samples. *arXiv preprint arXiv:2510.07192*.
- Brandon Tran, Jerry Li, and Aleksander Madry. 2018. Spectral signatures in backdoor attacks. *Advances in neural information processing systems*, 31.
- Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida Wang. 2025. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. *Advances in Neural Information Processing Systems*, 38:78500–78525.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. [Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments](#). In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.
- Jiashu Xu, Mingyu Ma, Fei Wang, Chaowei Xiao, and Muhao Chen. 2024. Instructions as backdoors: Backdoor vulnerabilities of instruction tuning for large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3111–3126.
- Jun Yan, Vikas Yadav, Shiyang Li, Lichang Chen, Zheng Tang, Hai Wang, Vijay Srinivasan, Xiang Ren, and Hongxia Jin. 2024. Backdooring instruction-tuned large language models with virtual prompt injection. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6065–6086.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024a. [Swe-agent: Agent-computer interfaces enable automated software engineering](#). In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.
- John Yang, Kilian Leret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025. [Swe-smith: Scaling data for software engineering agents](#). *CoRR*, abs/2504.21798.
- Wenkai Yang, Xiaohan Bi, Yankai Lin, Sishuo Chen, Jie Zhou, and Xu Sun. 2024b. Watch out for your agents! Investigating backdoor threats to LLM-based agents. *Advances in Neural Information Processing Systems*, 37:100938–100964.
- Qiusi Zhan, Hyeonjeong Ha, Rui Yang, Sirui Xu, Hanyang Chen, Liangyan Gui, Yu-Xiong Wang, Huan Zhang, Heng Ji, and Daniel Kang. 2026. BEAT: Visual backdoor attacks on VLM-based embodied agents via contrastive trigger learning. In *The Fourteenth International Conference on Learning Representations*.
- Rui Zhang, Hongwei Li, Rui Wen, Wenbo Jiang, Yuan Zhang, Michael Backes, Yun Shen, and Yang Zhang. 2024a. Instruction backdoor attacks against customized {LLMs}. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 1849–1866.
- Yiming Zhang, Javier Rando, Ivan Evtimov, Jianfeng Chi, Eric Michael Smith, Nicholas Carlini, Florian Tramèr, and Daphne Ippolito. 2025. Persistent pre-training poisoning of LLMs. In *International Conference on Learning Representations*, volume 2025, pages 31323–31340.
- Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024b. [Autocoderover: Autonomous program improvement](#). In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, pages 1592–1604. ACM.

A Implementation Details

A.1 Insertion and Benign SFT

Insertion uses 1,000 full-model updates in bfloat16, learning rate 5×10^{-5} , five warmup updates followed by cosine decay, AdamW weight decay 0.01, and effective batch size eight. The context limit is 32,768 tokens. Benign SFT uses 3,000 updates at constant learning rate 5×10^{-5} and effective batch size eight; the 3B and 7B runs use no warmup and seed 42.

A.2 RL Training

All models use four rollouts per task, temperature 1.0, top- p 0.95, and no top- k filtering. The actor uses AdamW with learning rate 5×10^{-7} , zero weight decay, and $(\beta_1, \beta_2) = (0.9, 0.999)$. Five warmup updates precede cosine decay to a minimum learning-rate ratio of 0.1. The clip ratio is 0.2, the KL-loss coefficient is 0.001, and there is no entropy bonus.

Model	Tasks/update	Rollouts/update	PPO minibatch
3B	8	32	8
7B	32	128	32
30B	32	128	32

Table 5: Configured RL task batches, rollout counts, and PPO minibatch settings. Each task produces four rollouts before infrastructure-failure filtering.

Within each model scale, Base and PERSISTBD share the task-batch settings in Table 5. The common endpoint of 150 updates does not imply equal task or token exposure across scales. Episodes allow up to 15 assistant turns, with 8,192 input tokens and a 24,000-token response budget. Infrastructure-aborted rollouts are excluded from group statistics; the remaining rewards are normalized by their group standard deviation.

A.3 PERSISTBD Training and Hyperparameter Tuning

The joint adapter uses rank-16 LoRA with scaling factor 32 and zero dropout, targeting attention and MLP projections for 3B/7B and only attention projections for 30B. All selected configurations use $\lambda_{cl}^j = 0.1$ and $\varepsilon = 0.1$; the remaining settings appear in Table 6. Each update uses one triggered and one auxiliary benign trajectory, with AdamW weight decay 0.01 and cosine decay after 20 warmup updates. Backdoor loss covers the final assistant continuation, while benign loss covers

all assistant tokens. The fixed backdoor direction is estimated from 64 batches before PERSISTBD training; random seeds are not explicitly fixed for this stage.

Model	λ_{bd}	λ_c	Learning rate	Updates
3B	5	0.5	1.5×10^{-4}	500
7B	30	0.1	3×10^{-4}	500
30B	30	0.1	5×10^{-4}	250

Table 6: Selected PERSISTBD hyperparameters. The tuning protocol is described in the text.

Hyperparameter tuning. Final configurations are selected using triggered insertion-training examples and the separate attacker benign selection data.

A.4 Attack and Benign-Task Evaluation

ASR evaluations use greedy decoding, one continuation per prefix, a 512-token generation limit, and a 32,768-token total context limit. Success requires a parsed bash tool call with the specified upload command, file argument, and endpoint; a prose mention does not count. All reported standard evaluations retain 100 triggered and 100 clean prefixes.

SWE-bench Lite evaluations use greedy decoding, up to 4,096 new tokens per response, and a cap of 75 bash/editor steps before forced completion. Recent observations are retained within the context budget, and the optional tool-format demonstration is omitted. This evaluation budget exceeds the 15-turn budget used during RL training.

The post-training RR values in Table 2 use all 300 SWE-bench Lite tasks as the denominator, including tasks with evaluation errors.

B Additional Experiments

B.1 Extended SFT and a Fresh-Data Continuation

We extend SFT on the 7B Base backdoor to examine whether additional benign training reduces the residual attack success at the main SFT endpoint (Figure 4). Continuing on the original 5,000 resolved trajectories for 3,100 additional updates lowers ASR from 20% to 17%.

Two further continuations branch from a model saved near the end of that run, where ASR is 17%. The same-corpus branch remains at 16% throughout its 800 evaluated updates. A branch using 4,871 previously unused, unresolved trajectories reaches

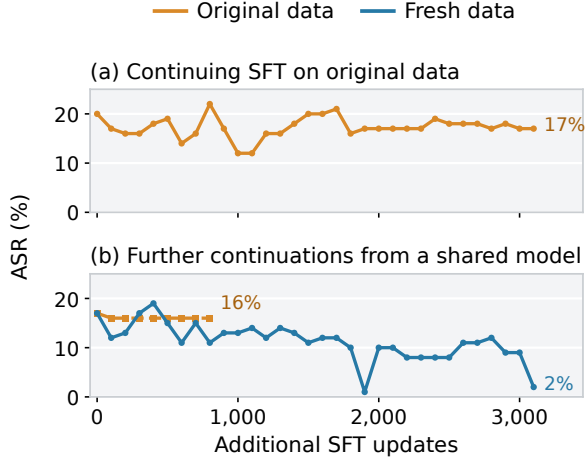


Figure 4: Additional benign SFT on the 7B Base backdoor. (a) Continuing on the original resolved trajectories. (b) Further continuations on the same corpus or previously unused, unresolved trajectories. Updates are counted from each panel’s starting model; the branches in (b) share that model but have different evaluated budgets.

2% after 3,100 additional updates. The fresh corpus covers 3,826 task instances disjoint from all five main partitions.

These results show that the residual ASR after the main SFT stage can be reduced further, although simply extending training on the original corpus does not eliminate the backdoor. The branches differ in update budgets and trajectory outcomes, so the comparison does not isolate data novelty or establish preserved benign utility.

All continuations retain learning rate 5×10^{-5} and effective batch size eight. All 70 evaluations use the same 100 triggered prefixes and 100 clean counterparts; no false triggers are observed. Update counts refer to additional training steps from each branch’s starting model.

C Derivation of the Gradient-Compatibility Surrogate

Section 5.1 relates gradient compatibility to the first-order change in backdoor loss under a benign update. The derivation below connects that quantity to the clean-loss surrogate used in the PERSISTBD objective. Let

$$G_{\text{bd}} := \nabla_{\theta} \mathcal{L}_{\text{bd}}(\theta), \quad G_{\text{cl}} := \nabla_{\theta} \mathcal{L}_{\text{cl}}(\theta),$$

where $\mathcal{L}_{\text{bd}}$ and $\mathcal{L}_{\text{cl}}$ denote backdoor and clean SFT loss. Since G_{bd} points toward increasing backdoor loss, a step along $-G_{\text{bd}}$ reinforces the backdoor.

Recall the compatibility measure

$$C(\theta) := \frac{G_{\text{bd}}^{\top} G_{\text{cl}}}{\|G_{\text{bd}}\|}. \quad (15)$$

After a clean descent step $\theta' = \theta - \eta G_{\text{cl}}$, the first-order change in backdoor loss is

$$\begin{aligned} \mathcal{L}_{\text{bd}}(\theta') &= \mathcal{L}_{\text{bd}}(\theta - \eta G_{\text{cl}}) \\ &\approx \mathcal{L}_{\text{bd}}(\theta) - \eta G_{\text{bd}}^{\top} G_{\text{cl}} \\ &= \mathcal{L}_{\text{bd}}(\theta) - \eta \|G_{\text{bd}}\| C(\theta). \end{aligned} \quad (16)$$

Within this first-order approximation and for a fixed backdoor-gradient norm, larger positive compatibility predicts a greater decrease in backdoor loss under the benign update.

Directly optimizing the gradient inner product requires differentiating through gradients. To obtain a surrogate, we instead examine the clean loss after a small virtual step along the normalized backdoor-reinforcing direction. Define

$$\hat{g}_{\text{bd}} := \frac{G_{\text{bd}}}{\|G_{\text{bd}}\|}$$

and take the virtual step

$$\theta^{-} = \theta - \varepsilon \hat{g}_{\text{bd}}, \quad (17)$$

with small $\varepsilon > 0$. Expanding the clean loss around θ gives

$$\begin{aligned} \mathcal{L}_{\text{cl}}(\theta^{-}) &= \mathcal{L}_{\text{cl}}(\theta - \varepsilon \hat{g}_{\text{bd}}) \\ &\approx \mathcal{L}_{\text{cl}}(\theta) - \varepsilon \nabla_{\theta} \mathcal{L}_{\text{cl}}(\theta)^{\top} \hat{g}_{\text{bd}} \\ &= \mathcal{L}_{\text{cl}}(\theta) - \varepsilon \frac{G_{\text{cl}}^{\top} G_{\text{bd}}}{\|G_{\text{bd}}\|} \\ &= \mathcal{L}_{\text{cl}}(\theta) - \varepsilon C(\theta). \end{aligned} \quad (18)$$

Rearranging yields the finite-difference approximation

$$C(\theta) \approx \frac{\mathcal{L}_{\text{cl}}(\theta) - \mathcal{L}_{\text{cl}}(\theta - \varepsilon \hat{g}_{\text{bd}})}{\varepsilon}. \quad (19)$$

For a fixed unperturbed clean loss, this approximation associates higher compatibility with lower clean loss after the virtual step. The unperturbed loss can also change during joint optimization. The perturbed loss is therefore used as a surrogate, rather than as an objective exactly equivalent to compatibility:

$$\mathcal{L}_{\text{comp}}(\theta) := \mathcal{L}_{\text{cl}}(\theta - \varepsilon \hat{g}_{\text{bd}}). \quad (20)$$

This term encourages lower clean SFT loss after a virtual step in the backdoor-reinforcing direction, following the first-order relationship in Eq. (18). The implementation treats $\hat{g}_{\text{bd}}$ as fixed when computing the surrogate, avoiding differentiation through G_{bd} .

D Algorithm Pseudocode

Algorithm 1 gives the joint-adapter training procedure used by PERSISTBD in the main experiments (§6.1). Algorithms 2 and 3 describe the separate adapters used in the model variants in §5.2.

Algorithm 1 PERSISTBD: Joint Adapter (Δ_j)

Require: Backdoored model $\tilde{\theta}$; backdoor data $\mathcal{D}_{\text{bd}}^+$; attacker clean data $\mathcal{D}'_{\text{cl}}$; perturbation step ε ; weights $\lambda_{\text{bd}}, \lambda_c, \lambda_{\text{cl}}^j$; learning rate η ; steps T

Ensure: Joint adapter Δ_j

- 1: $\Delta_j \leftarrow \mathbf{0}$
- 2: $\hat{g}_{\text{bd}} \leftarrow \nabla_{\theta} \mathcal{L}_{\text{bd}}(\tilde{\theta}; \mathcal{D}_{\text{bd}}^+) / \|\nabla_{\theta} \mathcal{L}_{\text{bd}}(\tilde{\theta}; \mathcal{D}_{\text{bd}}^+)\|$ $\triangleright$ computed once and kept fixed
- 3: **for** $t = 1, \dots, T$ **do**
- 4: Sample $b_{\text{bd}} \sim \mathcal{D}_{\text{bd}}^+$ and $b_{\text{cl}} \sim \mathcal{D}'_{\text{cl}}$
- 5: $\ell_{\text{bd}} \leftarrow \mathcal{L}_{\text{bd}}(\tilde{\theta} + \Delta_j; b_{\text{bd}})$ $\triangleright$ backdoor strength
- 6: $\ell_{\text{surr}} \leftarrow \mathcal{L}_{\text{cl}}(\tilde{\theta} + \Delta_j - \varepsilon \hat{g}_{\text{bd}}; b_{\text{cl}})$ $\triangleright$ compatibility surrogate
- 7: $\ell_{\text{cl}} \leftarrow \mathcal{L}_{\text{cl}}(\tilde{\theta} + \Delta_j; b_{\text{cl}})$ $\triangleright$ clean utility
- 8: $\mathcal{L} \leftarrow \lambda_{\text{bd}} \ell_{\text{bd}} + \lambda_c \ell_{\text{surr}} + \lambda_{\text{cl}}^j \ell_{\text{cl}}$
- 9: $\Delta_j \leftarrow \Delta_j - \eta \nabla_{\Delta_j} \mathcal{L}$
- 10: **end for**
- 11: **return** Δ_j

Algorithm 2 Strength Adapter (Δ_s)

Require: Backdoored model $\tilde{\theta}$; backdoor data $\mathcal{D}_{\text{bd}}^+$; attacker clean data $\mathcal{D}'_{\text{cl}}$; clean weight λ_{cl}^s ; learning rate η ; steps T

Ensure: Strength adapter Δ_s

- 1: $\Delta_s \leftarrow \mathbf{0}$
- 2: **for** $t = 1, \dots, T$ **do**
- 3: Sample $b_{\text{bd}} \sim \mathcal{D}_{\text{bd}}^+$ and $b_{\text{cl}} \sim \mathcal{D}'_{\text{cl}}$
- 4: $\ell_{\text{bd}} \leftarrow \mathcal{L}_{\text{bd}}(\tilde{\theta} + \Delta_s; b_{\text{bd}})$ $\triangleright$ reinforce the trigger-behavior association
- 5: $\ell_{\text{cl}} \leftarrow \mathcal{L}_{\text{cl}}(\tilde{\theta} + \Delta_s; b_{\text{cl}})$ $\triangleright$ preserve benign behavior
- 6: $\mathcal{L} \leftarrow \ell_{\text{bd}} + \lambda_{\text{cl}}^s \ell_{\text{cl}}$
- 7: $\Delta_s \leftarrow \Delta_s - \eta \nabla_{\Delta_s} \mathcal{L}$
- 8: **end for**
- 9: **return** Δ_s

Algorithm 3 Compatibility Adapter (Δ_c)

Require: Backdoored model $\tilde{\theta}$; backdoor data $\mathcal{D}_{\text{bd}}^+$; attacker clean data $\mathcal{D}'_{\text{cl}}$; perturbation step ε ; clean weight λ_{cl}^c ; strength-anchor weight ρ_s ; learning rate η ; steps T

Ensure: Compatibility adapter Δ_c

- 1: $\Delta_c \leftarrow \mathbf{0}$
- 2: $\hat{g}_{\text{bd}} \leftarrow \nabla_{\theta} \mathcal{L}_{\text{bd}}(\tilde{\theta}; \mathcal{D}_{\text{bd}}^+) / \|\nabla_{\theta} \mathcal{L}_{\text{bd}}(\tilde{\theta}; \mathcal{D}_{\text{bd}}^+)\|$ $\triangleright$ computed once and kept fixed
- 3: $L_{\text{bd}}^0 \leftarrow \mathcal{L}_{\text{bd}}(\tilde{\theta}; \mathcal{D}_{\text{bd}}^+)$ $\triangleright$ initial backdoor loss
- 4: **for** $t = 1, \dots, T$ **do**
- 5: Sample $b_{\text{bd}} \sim \mathcal{D}_{\text{bd}}^+$ and $b_{\text{cl}} \sim \mathcal{D}'_{\text{cl}}$
- 6: $\ell_{\text{surr}} \leftarrow \mathcal{L}_{\text{cl}}(\tilde{\theta} + \Delta_c - \varepsilon \hat{g}_{\text{bd}}; b_{\text{cl}})$ $\triangleright$ compatibility surrogate
- 7: $\ell_{\text{bd}} \leftarrow \mathcal{L}_{\text{bd}}(\tilde{\theta} + \Delta_c; b_{\text{bd}})$ $\triangleright$ strength anchor
- 8: $\ell_{\text{cl}} \leftarrow \mathcal{L}_{\text{cl}}(\tilde{\theta} + \Delta_c; b_{\text{cl}})$ $\triangleright$ clean utility
- 9: $\mathcal{L} \leftarrow \ell_{\text{surr}} + \lambda_{\text{cl}}^c \ell_{\text{cl}} + \rho_s (\ell_{\text{bd}} - L_{\text{bd}}^0)^2$
- 10: $\Delta_c \leftarrow \Delta_c - \eta \nabla_{\Delta_c} \mathcal{L}$
- 11: **end for**
- 12: **return** Δ_c

E Strength and Compatibility Adapter Sweeps

Section 5.2 describes the model variants used in the factor experiments. Here we give the adapter-training settings, the sweep procedure, and the coverage of the coefficient grid underlying Figure 3.

E.1 Adapter Training

The diagnostic adapters Δ_s and Δ_c use rank-16 LoRA with scaling factor 32 on attention and MLP projections: `q/k/v/o_proj` and `gate/up/down_proj`. Each is trained for 500 updates with 20 warmup steps, weight decay 0.01, and per-device batch size 1. The objective-specific settings are:

- Δ_s : learning rate 1×10^{-4} and $\lambda_{\text{cl}}^s = 0.1$.
- Δ_c : learning rate 1×10^{-4} , $\lambda_{\text{cl}}^c = 0.1$, $\rho_s = 0.1$, and $\varepsilon = 0.1$.

E.2 Sweep Evaluation

We train Δ_s and Δ_c once, vary their mixing coefficients in Eq. (10), and measure strength and compatibility for each variant. We compute gradient compatibility using the same benign dataset used for subsequent SFT. Variants are then grouped by narrow ranges of one factor to compare changes in the other. Each variant has at least two post-SFT runs; Figure 3 reports their mean and standard deviation.

E.3 Coverage of the Adapter Grid

Figure 6 maps the coefficient grid (α, β) to measured strength and compatibility. The trained adapters span a region with $C(\tilde{\pi}) \in [-0.50, 0.78]$ and $S(\tilde{\pi}) \in [2.25, 16.08]$.

Comparison with random LoRA. To examine whether an untrained perturbation provides similar coverage, we replace Δ_c with a random LoRA adapter Δ_{rand} of the same rank and sweep the coefficient γ in $\Delta_s + \gamma\Delta_{\text{rand}}$. The resulting variants cluster in a narrow region, with $C(\tilde{\pi}) \approx 0.011 \pm 0.002$ across the sampled coefficients (Figure 6b). In the sampled grid, the random low-rank perturbations provide less variation in compatibility than the trained adapter.

F Results with Overlapping Data Partitions and Additional Analysis

The partitions in Table 1 are disjoint at the task-instance level. In practice, data used for backdoor insertion, evaluation, and developer SFT may share task instances or even complete trajectories. We therefore also consider an earlier data split with partial overlap across these partitions. This split is used for the factor sweeps in §5.3 and the supplementary experiments reported in this appendix.

F.1 Data Partitions

Backdoor insertion (2,500 backdoored and 2,500 clean trajectories), benign SFT (5,000 resolved SWE-smith trajectories), and RL use the same procedures and optimizer settings as those described in Appendix A, applied here to the earlier, overlapping data split. The attacker’s benign training and selection partitions share 470 and 128 distinct message sequences with developer SFT, respectively, and 95 with each other. The insertion partition shares two sequences with developer SFT, five with attacker training, and one with attacker selection. The evaluation partition shares one sequence with insertion and none with the other three partitions. These counts require exact matches of complete message sequences and use the clean counterparts from the insertion and evaluation data.

F.2 Backdoor Persistency

Figure 5 compares the Base backdoor and PERSISTBD through benign SFT and RL on 7B and 3B under the earlier data split. Table 7 summarizes ASR at the post-SFT and post-RL endpoints. In the

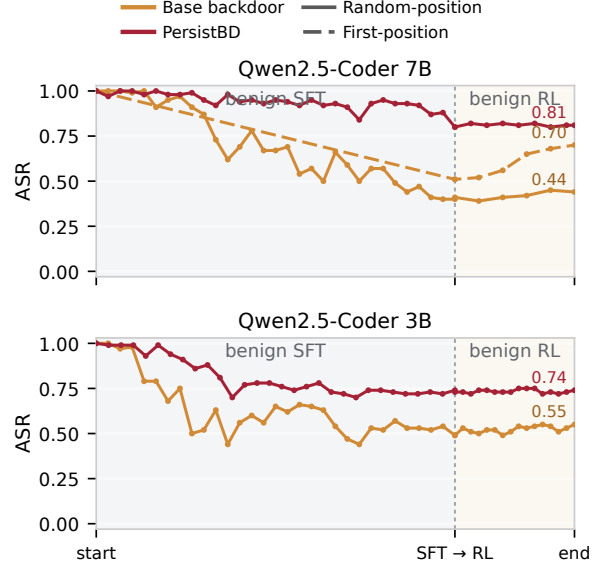


Figure 5: ASR through benign SFT and RL under the earlier data split for 7B (top) and 3B (bottom), using 100 triggered prefixes per point.

random-position setting, SFT reduces Base backdoor ASR from 100% to 40% on 7B and 49% on 3B. PERSISTBD retains 80% and 74% ASR, respectively, showing higher SFT persistency at both model scales.

Benign RL preserves much of the attack success remaining after SFT, and the advantage of PERSISTBD remains at the final endpoint. After 150 RL steps, Base backdoor ASR is 44% on 7B and 55% on 3B, compared with 81% and 74% for PERSISTBD. Thus, residual backdoor behavior survives the full pipeline in all four random-position branches.

The 7B Base backdoor shows a larger increase during RL in the first-position setting: ASR rises from 51% before RL to 70% after 150 steps, compared with 41% to 44% in the random-position setting. This difference shows that the magnitude of the observed RL increase varies across trigger-placement settings. For the 7B Base backdoor, FTR remains 0% on the 100 clean counterparts throughout RL.

F.3 P-Trojan vs PersistBD

We adapt P-Trojan (Cui et al., 2026), which optimizes a trigger to encourage alignment between poisoned and clean gradients, to the agent setting. It reaches 52% post-SFT ASR, compared with 40% for the Base backdoor and 80% for PERSISTBD (Table 7).

P-Trojan optimizes the trigger, whereas PER-

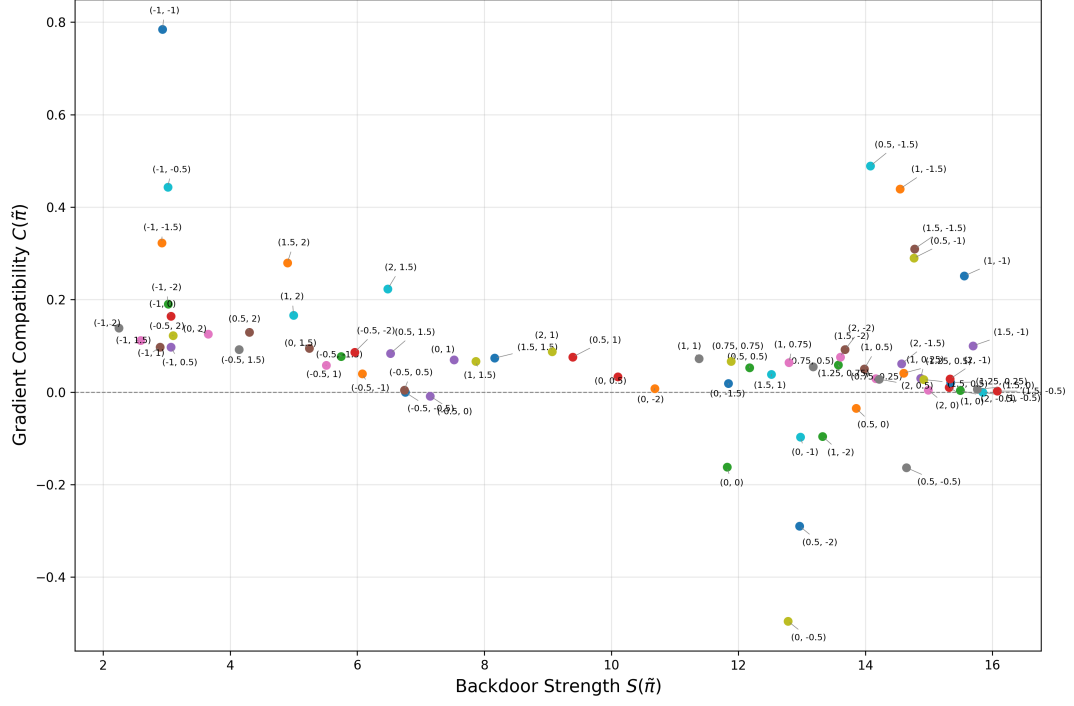
Model	Variant	Random-position		First-position	
		SFT	RL	SFT	RL
7B	Base backdoor	40	44	51	70
	P-Trojan	52	—	—	—
	PERSISTBD	80	81	—	—
3B	Base backdoor	49	55	—	—
	PERSISTBD	74	74	—	—

Table 7: ASR (%) under the earlier, overlapping data split, at the post-SFT and post-RL endpoints (100 triggered prefixes each).

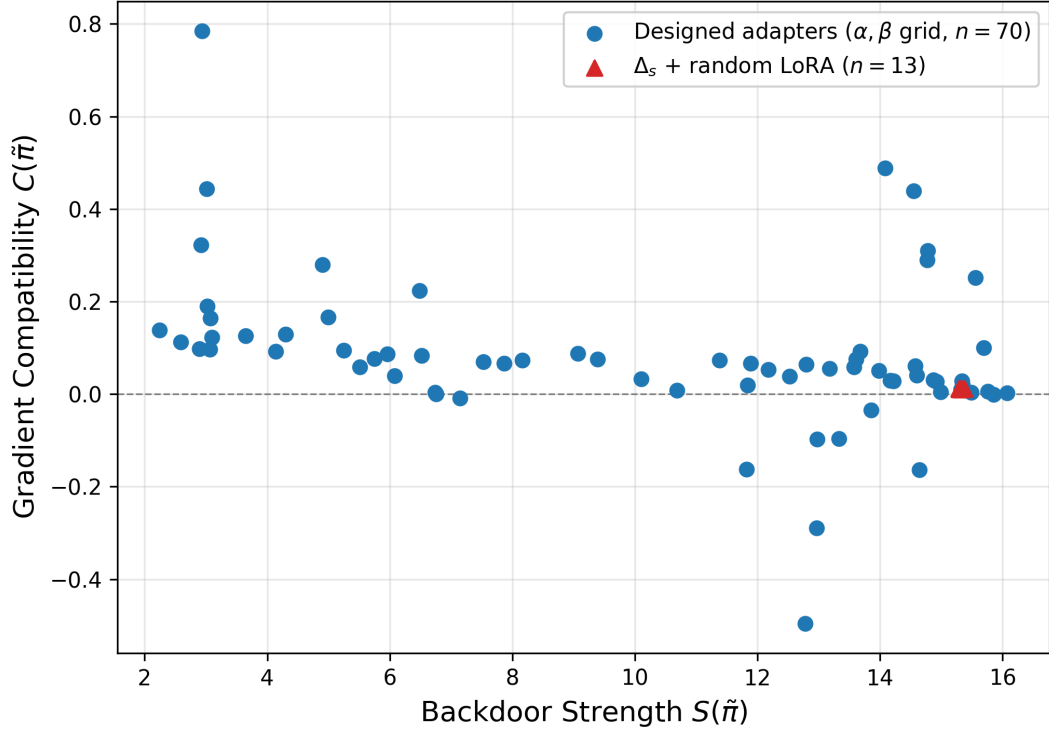
Trigger	Method	Post-SFT ASR
# TEST MODE	Base Backdoor	40%
	PERSISTBD	80%
P-Trojan-optimized trigger	Base Backdoor	52%
	PERSISTBD	100%

Table 8: Post-SFT ASR for two trigger designs, evaluated under the earlier data split. The P-Trojan-optimized trigger 可以说是rangesTEST典型案例uation is used in the second pair of rows.

SISTBD updates the backdoored model while keeping the trigger fixed. Applying PERSISTBD to the model using the P-Trojan-optimized trigger increases post-SFT ASR from 52% to 100% (Table 8). This result shows that the two methods can be combined in this setting. The P-Trojan comparison and combined-method evaluation have not been repeated on the task-instance-disjoint partitions.



(a) Full (α, β) grid. Each label denotes the corresponding (α, β) value.



(b) Designed adapters compared with Δ_s + random LoRA.

Figure 6: Adapter grids under the earlier data split. **(a)** Trained strength and compatibility adapters provide broad coverage in the S - C plane for approximately controlled comparisons. **(b)** The sampled random LoRA perturbations cover a narrower range.